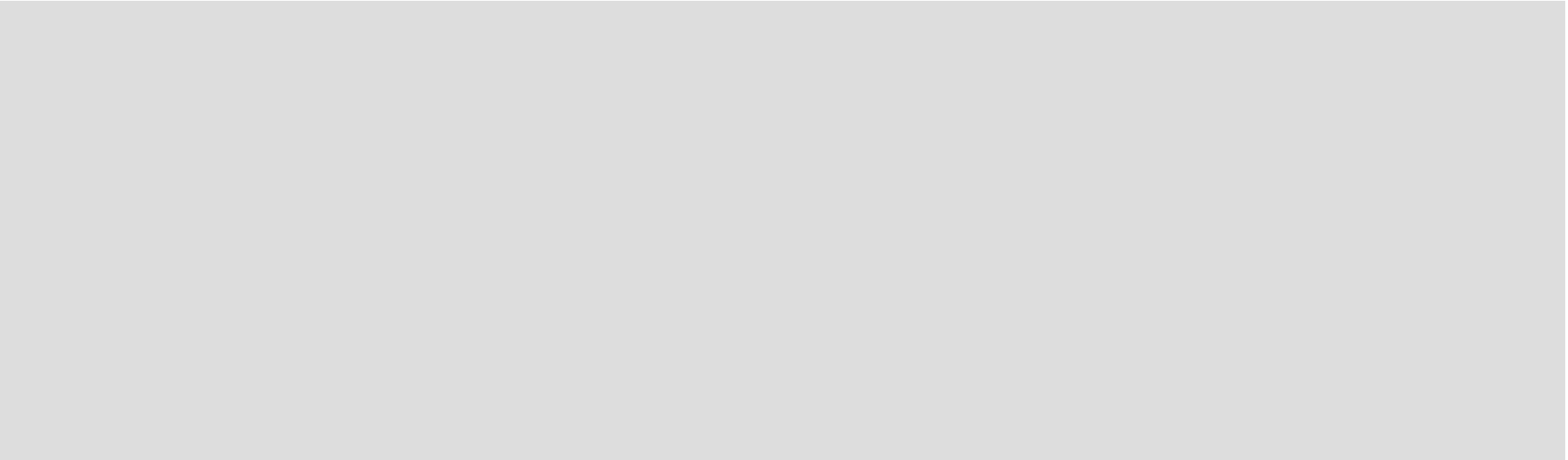

PARALLEL AND DISTRIBUTED COMPUTING

CHAPTER 7: OPERATING SYSTEM SUPPORT



LESSON TOPICS

- Introduction
- The operating system layer
- Protection
- Processes and threads
- Communication and invocation
- Operating system architecture
- Virtualization at the operating system level

PROCESS AND THREADS

- A thread is the operating system abstraction of an activity (“thread of execution”)
- An execution environment is the unit of resource management: a collection of local kernel-managed resources to which its threads have access
- An execution environment primarily consists
 - An address space
 - Thread synchronization and communication resources such as semaphore and communication interfaces
 - High-level resources such as open file and windows

ADDRESS SPACES

- Region, separated by inaccessible areas of virtual memory
- Region do not overlap
- Each region is specified by the following properties
 - Its extent (lowest virtual address and size)
 - Read/write/execute permissions for the process's threads
 - Whether it can be grown upwards or downward

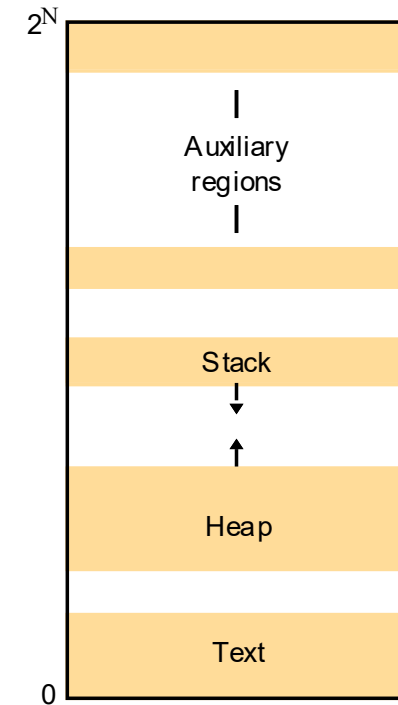


Figure 7.3 Address space

ADDRESS SPACES

- A mapped file is one that is accessed as an array of bytes in memory. The virtual memory system ensures that accesses made in memory are reflected in the underlying file storage
- A shared memory region is that is backed by the same physical memory as one or more regions belonging to other address spaces
- The uses of shared regions include the following
 - Libraries
 - Kernel
 - Data sharing and communication

CREATION OF A NEW PROCESS

- The creation of a new process has been an indivisible operation provided by the operating system. For example, the UNIX *fork* system call.
- For a distributed system, the design of the process creation mechanism has to take account of the utilization of multiple computers
- The choice of a new process can be separated into two independent aspects
 - The choice of a target host
 - The creation of an execution environment

CHOICE OF PROCESS HOST

- The choice of node at which the new process will reside – the process allocation decision – is a matter of policy
- Transfer policy
 - Determines whether to situate a new process locally or remotely. For example, on whether the local node is lightly or heavily load
- Location policy
 - Determines which node should host a new process selected for transfer. This decision may depend on the relative loads of nodes, on their machine architectures and on any specialized resources they may process

CHOICE OF PROCESS HOST

- Process location policies may be

- Static
- Adaptive

Load manager collect information about the nodes and use it to allocate new processes to node

- Load-sharing systems may be

- Centralized
- Hierarchical
- Decentralized

One load manager component

Several load manager organized in a tree structure

Node exchange information with one another direct to make allocation decisions

CHOICE OF PROCESS HOST

- In sender-initiated load-sharing algorithms, the node that requires a new process to be created is responsible for initiating the transfer decision
- In receiver-initiated algorithm, a node whose load is below a given threshold advertises its existence to other nodes so that relatively loaded nodes will transfer work to it
- Migratory load-sharing systems can shift load at any time, not just when a new process is created. They use a mechanism called process migration

CREATION OF NEW EXECUTION ENVIRONMENT

- There are two approaches to defining and initializing the address space of a newly created process
- Where the address space is of statically defined format
 - For example, it could contain just a program text region, heap region and stack region
 - Address space regions are initialized from an executable file or filled with zeroes as appropriate
- The address space can be defined with respect to an existing execution environment
 - For example, the newly created child process physically shares the parent's text region, and has heap and stack regions that are copies of the parents in extent (as well as in initial contents)
 - When parent and child share a region, the page frames belonging to the parent's region are mapped simultaneously into the corresponding child region

CREATION OF NEW EXECUTION ENVIRONMENT

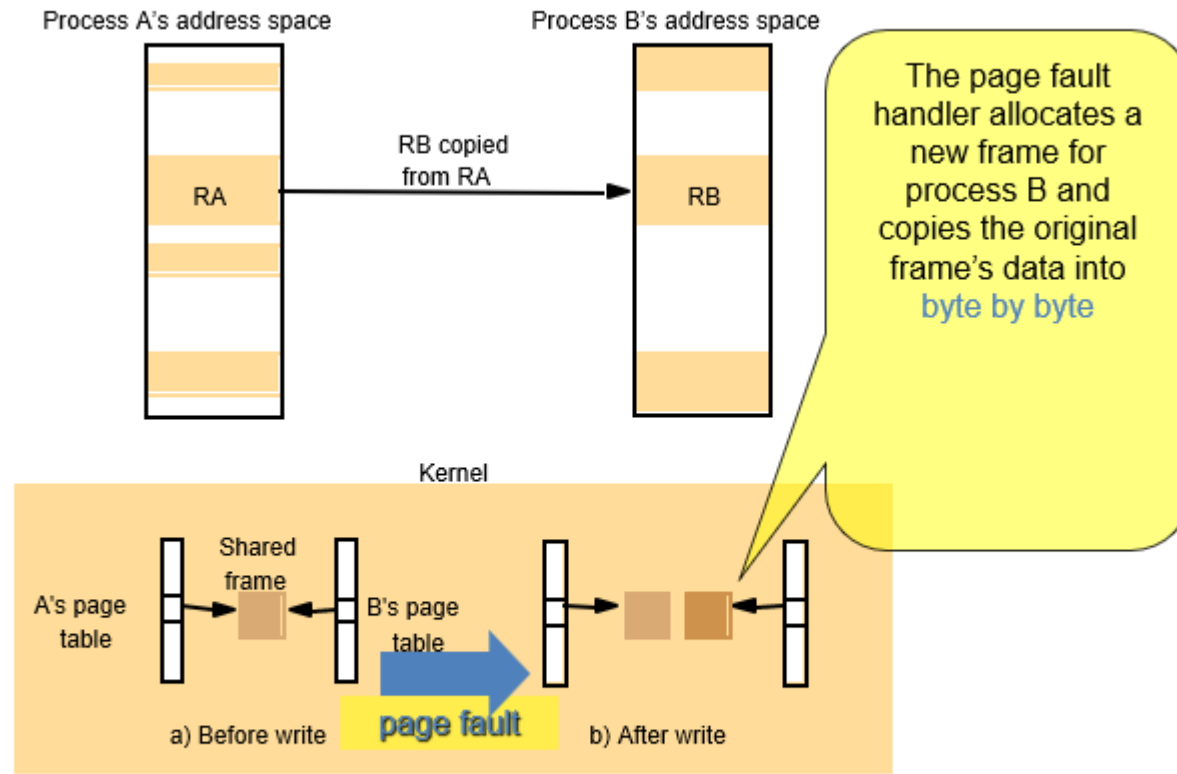


Figure 7.4 Copy-on-write

THREADS

- The next key aspect of a process to consider in more detail and server process to possess more than one thread.

CLIENT AND SERVER WITH THREADS

Worker pool

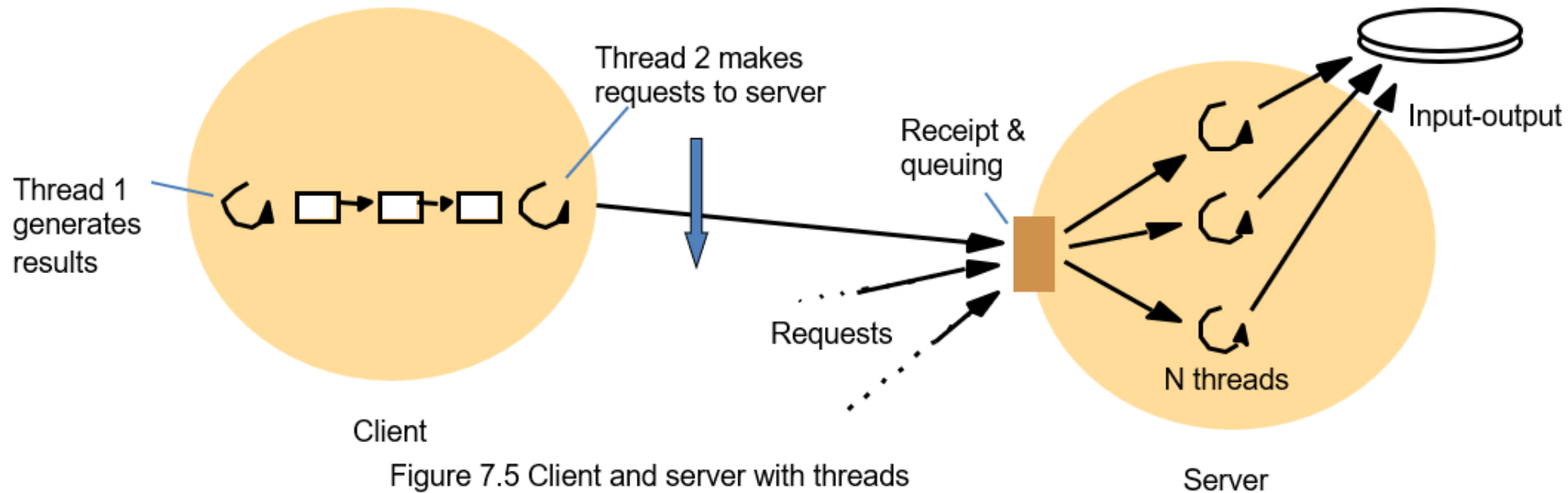


Figure 7.5 Client and server with threads

A disadvantage of this architecture is its inflexibility

Another disadvantage is the high level of switching between the I/O and worker threads as they manipulate the share queue

ALTERNATIVE SERVER THREADING ARCHITECTURES

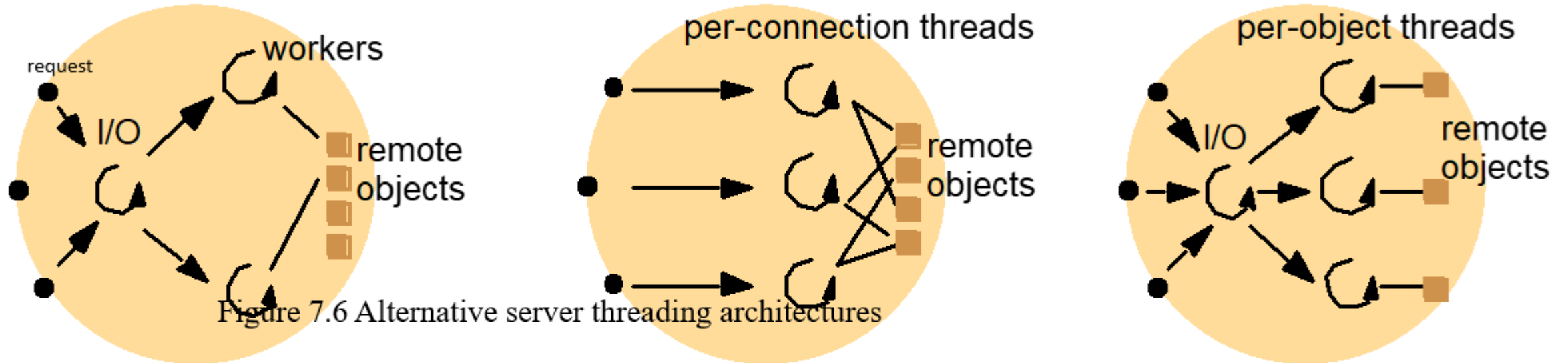


Figure 7.6 Alternative server threading architectures

a. Thread-per-request

b. Thread-per-connection

c. Thread-per-object

Advantage: the threads do not contend for a shared queue, and throughput is potentially maximized

Disadvantage: the overheads of the thread creation and destruction operations

In each of these last two architectures the server benefits from lowered thread-management overheads compared with the thread-per-request architecture.

Their disadvantage is that clients may be delayed while a worker thread has several outstanding requests but another thread has no work to perform

STATE ASSOCIATED WITH EXECUTION ENVIRONMENT AND THREADS

<i>Execution environment</i>	<i>Thread</i>
Address space tables	Saved processor registers
Communication interfaces, open files	Priority and execution state (such as <i>BLOCKED</i>)
Semaphores, other synchronization objects	Software interrupt handling information
List of thread identifiers	Execution environment identifier
Pages of address space resident in memory; hardware cache entries	

A COMPARISON OF PROCESSES AND THREADS AS FOLLOW

- Creating a new thread with an existing process is cheaper than creating a process.
- More importantly, switching to a different thread within the same process is cheaper than switching between threads belonging to different process.
- Threads within a process may share data and other resources conveniently and efficiently compared with separate processes.
- But, by the same token, threads within a process are not protected from one another.

A COMPARISON OF PROCESSES AND THREADS AS FOLLOW

- The overheads associated with creating a process are in general considerably greater than those of creating a new thread.
 - A new execution environment must first be created, including address space table
- The second performance advantage of threads concerns switching between threads – that is, running one thread instead of another at a given process
- A context switch is the transition between contexts that takes place when switching between threads, or when a single thread makes a system call or takes another type of exception
- It involves the following:
 - The saving of the processor's original register state, and loading of the new state
 - In some cases; a transfer to a new protection domain – this is known as a domain transition

THREAD SCHEDULING

- In preemptive scheduling, a thread may be suspended at any point to make way for another thread
- In non-preemptive scheduling, a thread runs until it makes a call to the threading system (for example, a system call).
- The advantage of non-preemptive scheduling is that any section of code that does not contain a call to the threading system is automatically a critical section
 - Race conditions are thus conveniently avoided
- Non-preemptively scheduled threads cannot take advantage of multiprocessor, since they run exclusively

THREAD IMPLEMENTATION

- When no kernel support for multi-thread process is provided, a user-level threads implementation suffers from the following problems
 - The threads with a process cannot take advantage of a multiprocessor
 - A thread that takes a page fault blocks the entire process and all threads within it
 - Threads within different processes cannot be scheduled according to a single scheme of relative prioritization

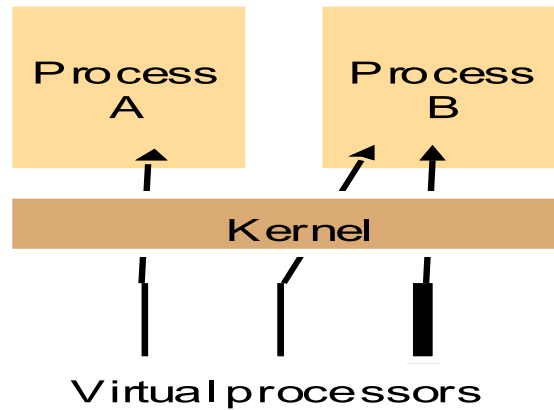
THREAD IMPLEMENTATION

- User-level threads implementations have significant advantages over kernel-level implementations
 - Certain thread operations are significantly less costly
 - For example, switching between threads belonging to the same process does not necessarily involve a system call – that is, a relatively expensive trap to the kernel
 - Given that the thread-scheduling module is implemented outside the kernel, it can be customized or changed to suit particular application requirements. Variations in scheduling requirements occur largely because of application-specific considerations such as the real-time nature of a multimedia processing
 - Many more user-level threads can be supported than could reasonably be provided by default by a kernel

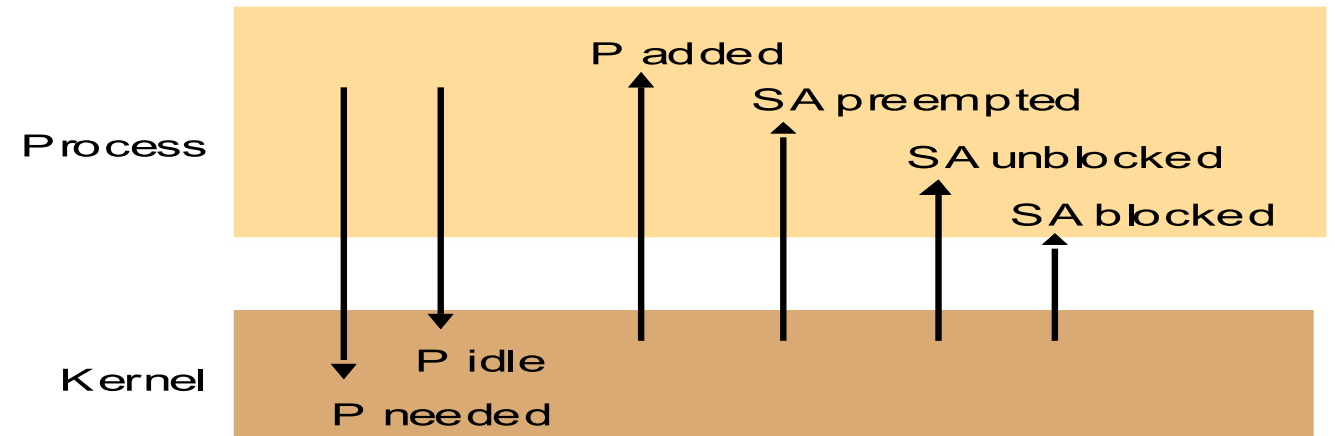
THE FOUR TYPE OF EVENT THAT KERNEL NOTIFIED TO THE USER LEVEL SCHEDULER

- Virtual processor allocated
 - The kernel has assigned a new virtual processor to the process, and this is the first time slice upon it; the scheduler can load the SA with the context of a READY thread, which can thus recommence execution
- SA blocked
 - An SA has blocked in the kernel, and kernel is using a fresh SA to notify the scheduler: the scheduler sets the state of the corresponding thread to BLOCKED and can allocate a READY thread to the notifying SA
- SA unblocked
 - An SA that was blocked in the kernel has become unblocked and is ready to execute at user level again; the scheduler can now return the corresponding thread to READY list. In order to create the notifying SA, the another SA in the same process. In the latter case, it also communicates the preemption event to the scheduler, which can re-evaluate its allocation of threads to SAs.
- SA preempted
 - The kernel has taken away the specified SA from the process (although it may do this to allocate a processor to a fresh SA in the same process); the scheduler places the preempted thread in the READY list and re-evaluates the thread allocation.

SCHEDULER ACTIVATIONS



A. Assignment of virtual processors to processes



B. Events between user-level scheduler & kernel
Key: P = processor; SA = scheduler activation

Scheduler activation (SA) is a call from kernel to a process

Figure 7.10 Scheduler activations

VIRTUALIZATION AT THE OPERATING SYSTEM LEVEL

- System Virtualization
- Virtual Machines
- Virtualization Architecture
- Hypervisor

SYSTEM VIRTUALIZATION



System Virtualization is to provide

- multiple virtual machines (virtual hardware images) over the underlying physical machine architecture,
- with each virtual machine running a separate operating system instance.

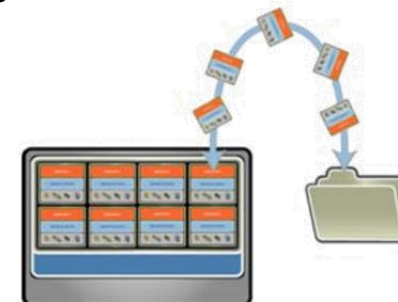
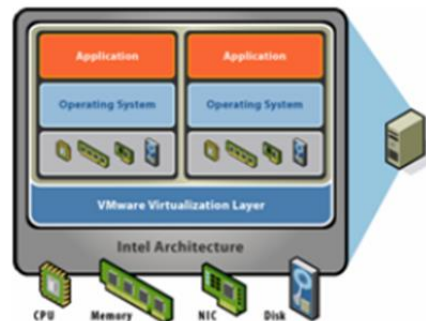
MOTIVATION FOR VIRTUALIZATION AT OS LEVEL

- ❖ Virtual machines can be migrated quite simply to other physical Machines
- ❖ Virtualization is very relevant to the provision of *cloud computing*; allowing users of the cloud to be provided with one or more virtual machines for their own use
- ❖ the need for distributed applications to create and destroy virtual machines readily and with little overhead
- ❖ Virtualization can be used to provide multiple operating system types on one physical architecture.

VIRTUAL MACHINES

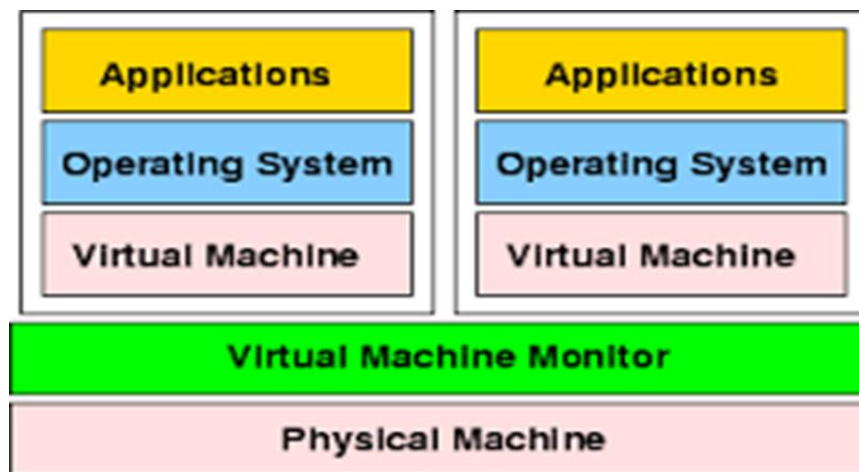
Virtual machines provide:

- ❖ Hardware independence – Guest VM sees the same hardware regardless of the host hardware
- ❖ Isolation – VM's operating system is isolated from the host operating system
- ❖ Encapsulation – Entire VM encapsulated into a single file



VIRTUALIZATION ARCHITECTURE

- A Virtual Machine (VM) is an isolated runtime environment (guest OS and applications)
- Multiple virtual systems (VMs) can run on a single physical system



HYPERVISOR

- A hypervisor, is a virtual machine manager/monitor (VMM), or virtualization manager, is a program that allows multiple operating systems to share a single hardware host.
- Two types of hypervisor
 - (i) Hosted Hypervisor
 - (ii) Bare-Metal Hypervisor

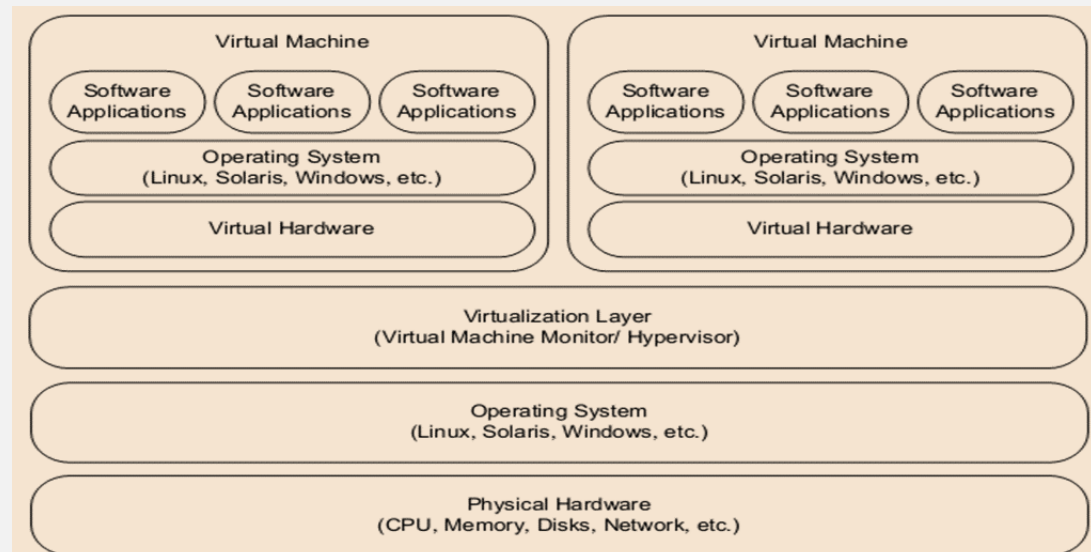
HYPERVISOR

1) Hosted

2) Bare-Metal

- The operating system has a virtualization platform installed into which one or more virtual machines are created where each can act as its own set of separate hardware and are capable of having an operating system and applications installed.

E.g. VMware Workstation



HYPERVISOR

1) Hosted

2) Bare-Metal

- Virtualized computing system is one in which the virtualization platform is installed directly onto the computer's hardware or installed.

E.g. Xen Server, VMware ESX/ESXi

